

A/Bテストをアップデートする

Takeshi Teshima

A/Bテストをアップデートする

今回は……

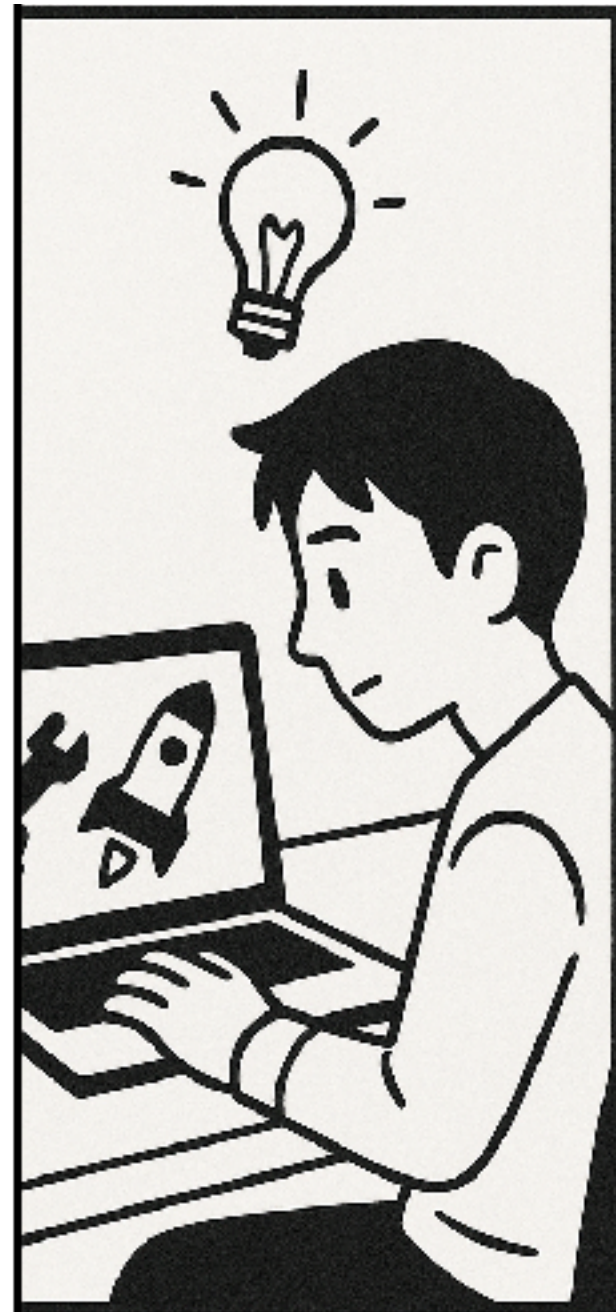
皆さんの**A/Bテストのサンプルサイズを**

平均で**7割ほど**に削減できる**話**かもしれない

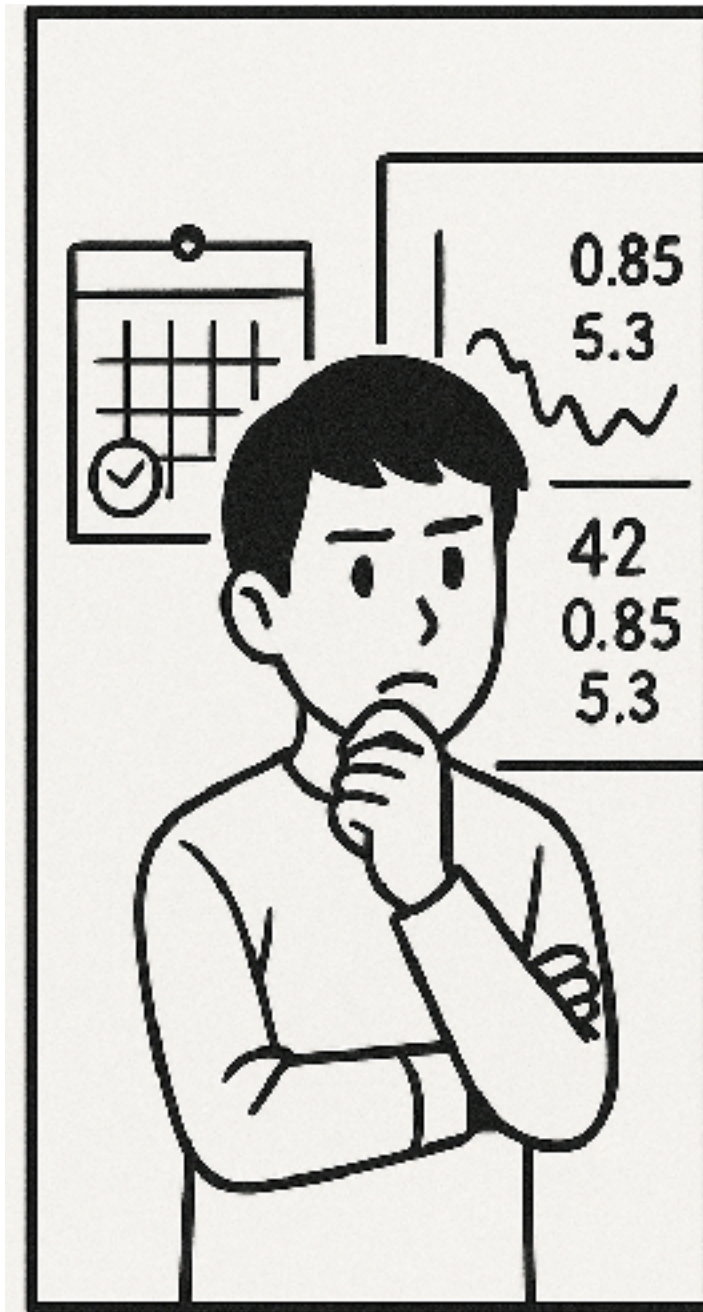
こんなことはありませんか？



A/Bテスト，途中で止めてしまいたい……



A/Bテストをするぞ！
打ち手の筋が良さそうで，
高い効果が望めそう



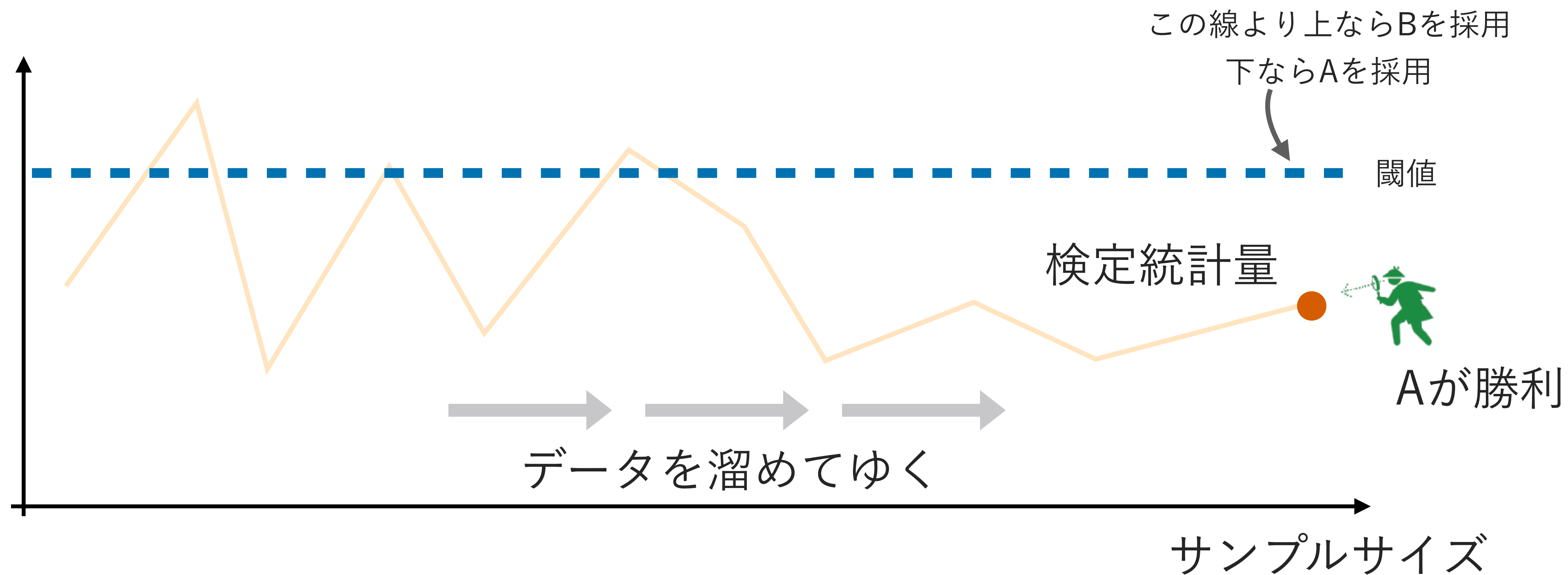
サンプルサイズは……
サンプル不足は避けたいから
少し保守的に設定しておくか



「初速良かったです！」
早めに全体展開したいなあ……
結果が出るのはXX週間後か……

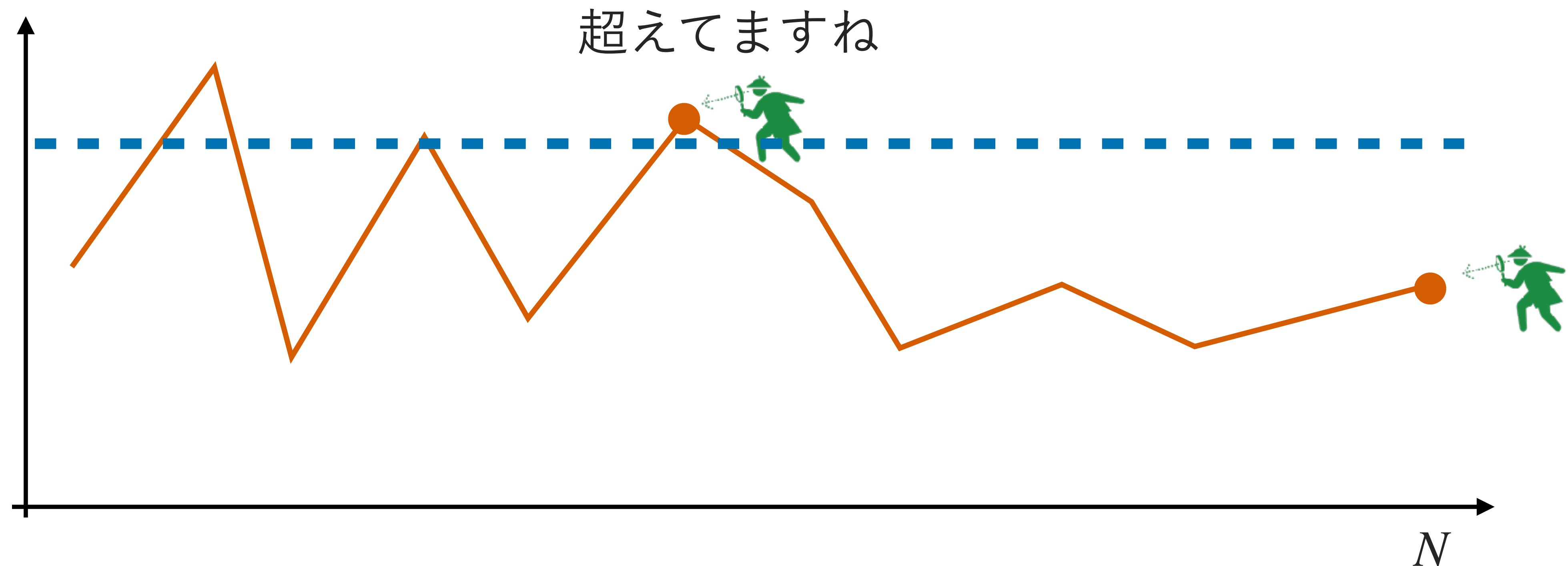
検定の枠組み

本来の検定では、データが全部出揃ってから統計量を計算・判定



検定の枠組み > 途中で検定してはいけない問題

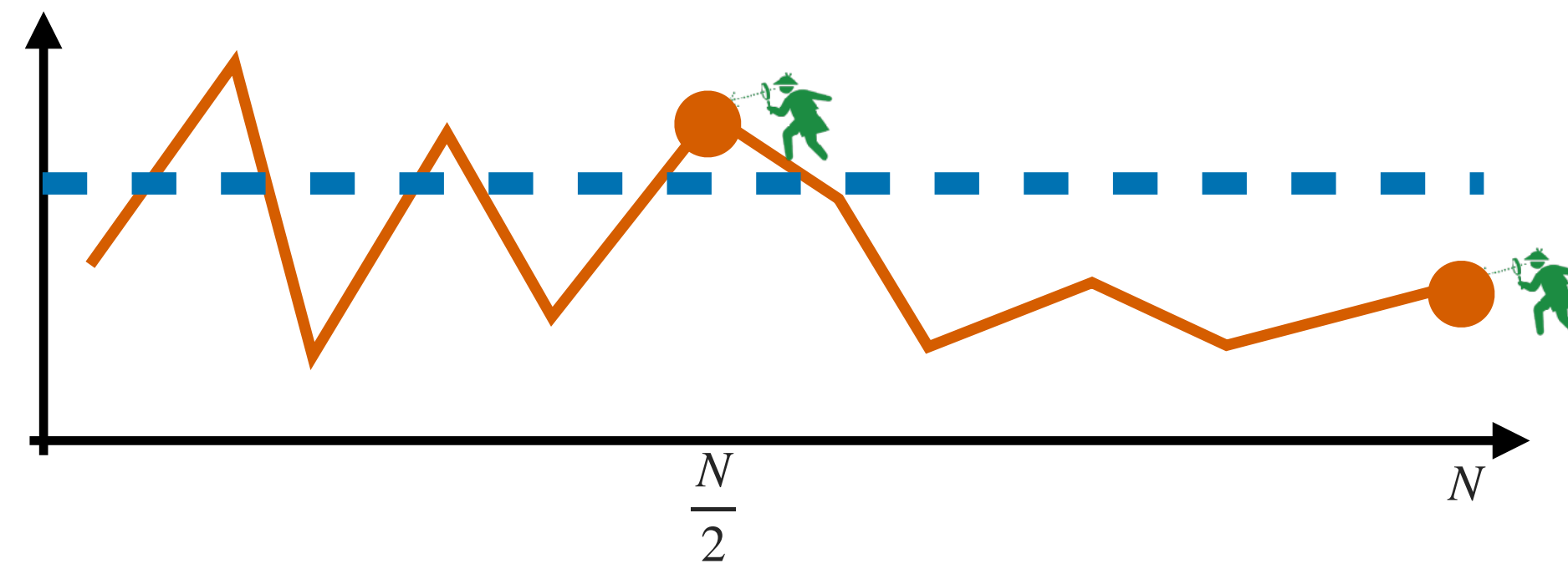
当然，調子が良いタイミングで好き勝手に検定をして良いわけではない



検定の枠組み > 途中で検定してはいけない問題

何が問題か？ → 多重検定になり，第一種の過誤率（偽陽性率）が増大

- 棄却できる機会が増えてしまうため，偽陽性が増える

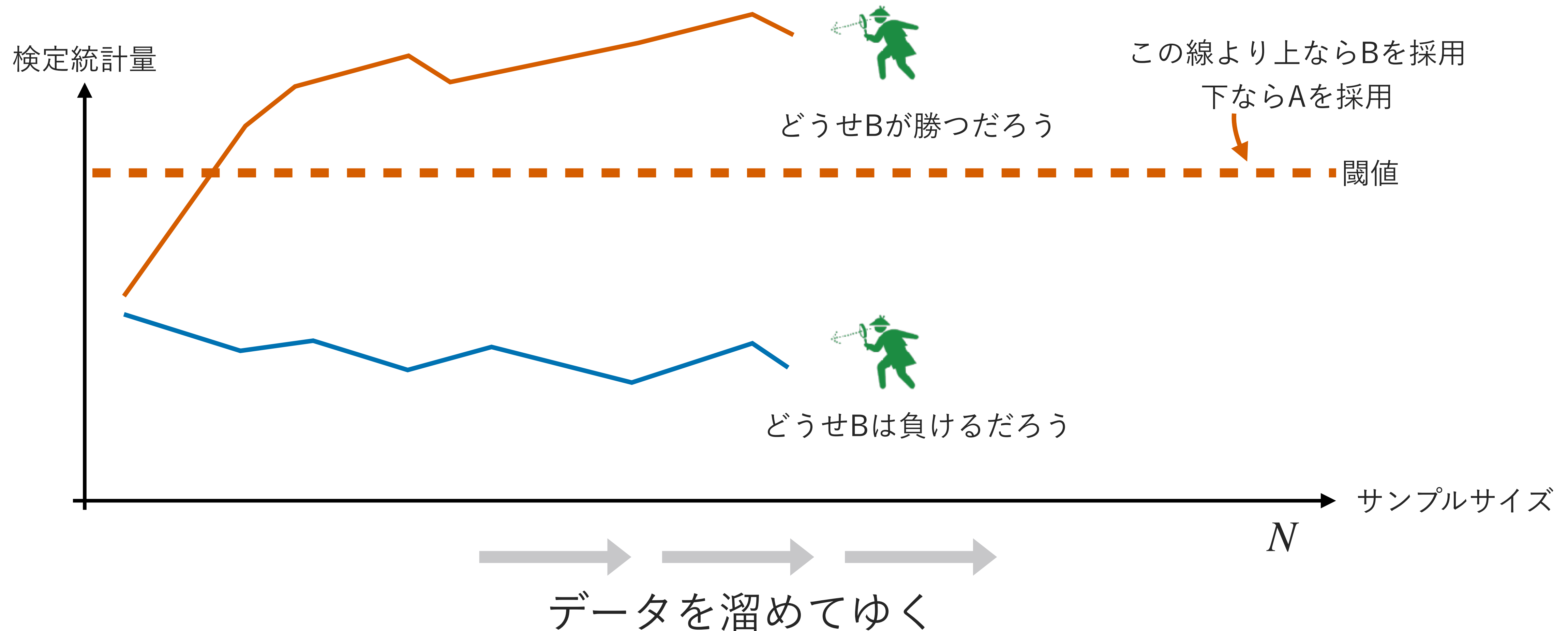


- 例) 1回だけ実行する前提で $\mathbb{P}_{H_0}(\text{reject}) \leq 0.05$ となるよう設計された検定でも……
 - 同じ検定を等間隔に2回行くと， $\mathbb{P}_{H_0}(\text{reject}) = 0.083$ に膨張するなど
 - せっかく統計的検定をしても，本来の設計から外れた結果になってしまう

【補足】「早めに検定をして，有意差が出なかったとしても，きちんと中止する」ことができるならば問題ない。しかし，有意差が出なかった場合に「もっとデータを溜めよう」と言って実験を継続してしまうと，この一連の手続きの中で，第一種の過誤の確率が高まってしまう。

検定の枠組み > 途中で検定してはいけない問題

そうはいっても、明らかに結論が見えているなら、早めに止めてしまいたい



検定の枠組み＞早めの検定をする方法



実は、早めの検定をして、サンプルサイズを減らせる方法があるんです！

- 著名な統計学者であるAbraham Waldらが1943年頃に開発！
- NetflixやSpotifyなど、大手企業も導入！
- 逸話
 - 戦時中にWaldが開発したが、軍需工場の検品工程で平均サンプルサイズを7割や5割に削減できるなど、あまりに高い効果に、アメリカの軍事的優位性に繋がる統計手法だとして当時は国家機密に指定されたい

【参考】 Wald, A. (1945). Sequential tests of statistical hypotheses. The Annals of Mathematical Statistics, 16(2), 117–186.

【補足】 Wald自身はハンガリー出身であったため、当初は機密に指定されてしまった自身の研究成果へのアクセス権がなかった、という皮肉な逸話も残っている。

【補足】 今回紹介する手法は、Waldが当初開発したものとは少し異なる。

検定の枠組み＞早めの検定をする方法



サンプルサイズを減らせる……胡散臭いな？

- 検定のサンプルサイズは、**最低限の必要な件数**を設計しているはず
- でも、サンプルサイズを平均で何割も減らせる方法がある……どうということ？

検定の枠組み＞早めの検定をする方法



ポイントは「**逐次的にデータが集まる**」という**構造**を利用すること

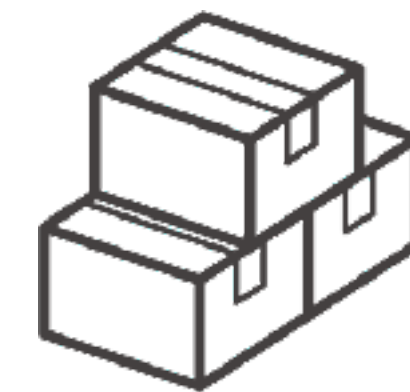
- 例) 最も極端な場合
 - **最後の1件のデータを収集する時点**では、既に勝敗が確定していることがある
 - 最後の1件が生じうる統計量の振れ幅が 0.1 で、すでに閾値よりも10を超えているならば、残り1件の結果が何であれ、棄却はもう決まっている
- 今回紹介する手法も、逐次性を利用することで何かを実現するという点は同じ（上とは異なる）

検定の枠組み＞早めの検定をする方法

A/Bテストなどでは、データが逐次的に収集されるので、途中で何かをできる

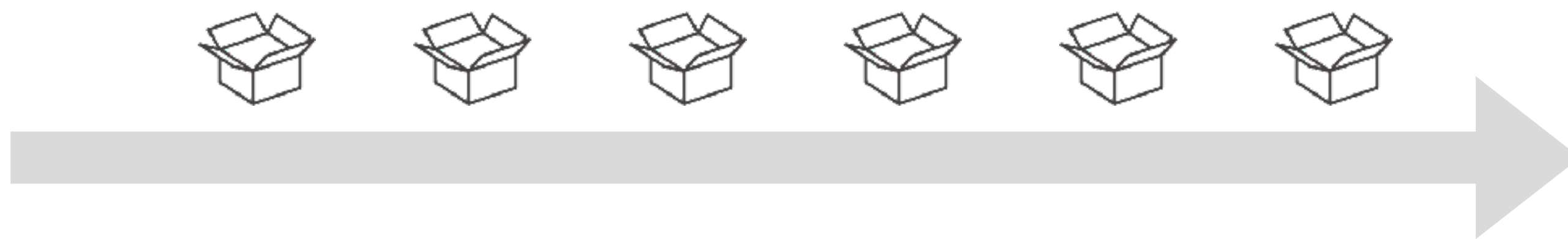
固定サンプル検定

一回でまとまって届く



逐次検定

徐々にデータが届く



【補足】統計的検定を発展・普及させたFisherは、英国のRothamsted農業試験場に勤務していた。農業試験では「検定開始から長時間経過して初めて結果が分かる」問題設定が自然であり、これが徐々にデータが蓄積される応用分野との違いとなっている。Armitage (1993, p. 392)は、「Fisherが医学や産業分野での研究に従事していたら統計的理論は違う発展を遂げていたかもしれないと考えると面白い」と述べている。

検定の枠組み＞早めの検定をする方法

広義に逐次検定（sequential testing）と呼ばれる手法群が色々ある

- 群逐次検定（group sequential testing）による中間解析（interim analyses）
 - **今回はこちらを説明**
 - 途中で何回かに分けて検定を実行する手法
 - 主目的とする指標の検定に適する
- 安全検定（safe tests）による随時正当な推論（anytime-valid inference）
 - 帰無仮説に反する証拠を継続的に蓄積する手法
 - 期間を決めずに指標を定常監視できるので、ガードレール指標と相性が良い

群逐次検定による中間解析 ～A/Bテストの早期終了～

これまでのあらすじ

- 臨床試験やA/Bテストなどでは、データを全部集めてから検定するより、途中で結果が明白なら早く止めたい。
 - 中間解析による早い全体展開（efficacy）あるいは無駄の抑制（futility）
- ただし、何も考えずに何度も検定をすると、第一種の過誤（偽陽性）のリスクが積み上がってしまう。
- 群逐次検定（group sequential test）は、この問題を制御しつつ中間解析を可能にする仕組み。

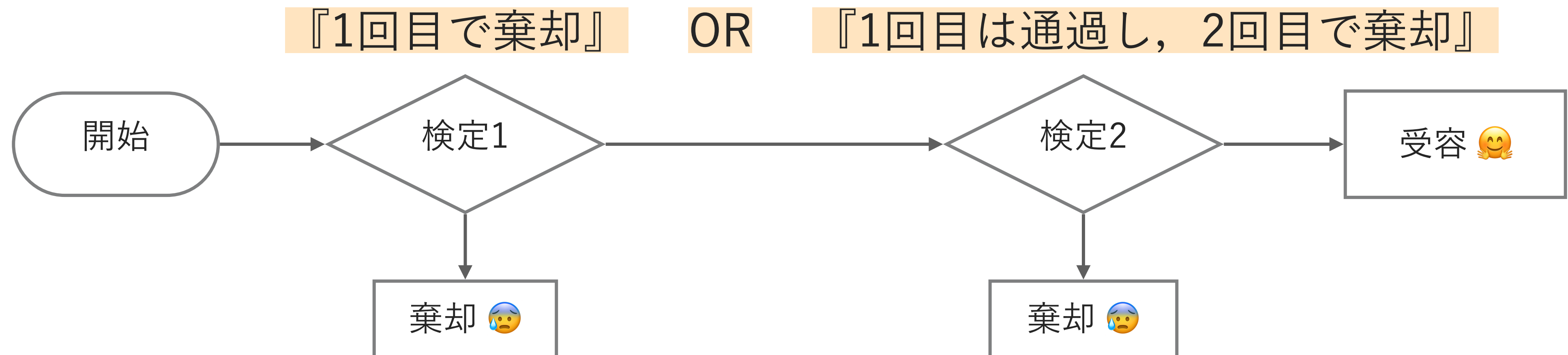
群逐次検定による中間解析＞基本の考え方

まずは、簡単のため、合計2ステージの検定（中間1回、最終1回）で説明する

- 第一種の過誤率は、帰無仮説の分布のもとで、棄却してしまう確率のことだった

$$\mathbb{P}_0(\text{reject})$$

- 今の場合、棄却されうるタイミングが2回ある



群逐次検定による中間解析＞基本の考え方

この場合の検定の多重性は、次の構造になっている

- OR条件での検定になっている

「早めの検定で有意差あり」 OR 「最終検定まで到達して、かつ有意差あり」

- この場合、第一種の過誤（偽陽性）のリスクは

- 本来の1回の検定の場合： $\mathbb{P}_{H_0}$ （最終検定で有意差）

- 2回検定する場合： $\mathbb{P}_{H_0}$ （早めの検定で有意差 OR 最終検定で有意差）

- もちろん（上式） $\leq$ （下式）なので、確かに第一種の過誤率が上昇している

【補足】細かく言えば、 $\mathbb{P}(\text{早めの検定で有意差 or (早めの検定で有意差なし and 最終検定で有意差)}) = \mathbb{P}(\text{早めの検定で有意差 or 最終検定で有意差})$ という理由で左辺の通りとなっている。

群逐次検定による中間解析＞基本の考え方

どうするか？検定の手続きを正しく考慮に入れればよい！

- OR条件での検定になっているので,

$$\mathbb{P}_{H_0}(\text{早めの検定で有意差 OR 最終検定で有意差})$$

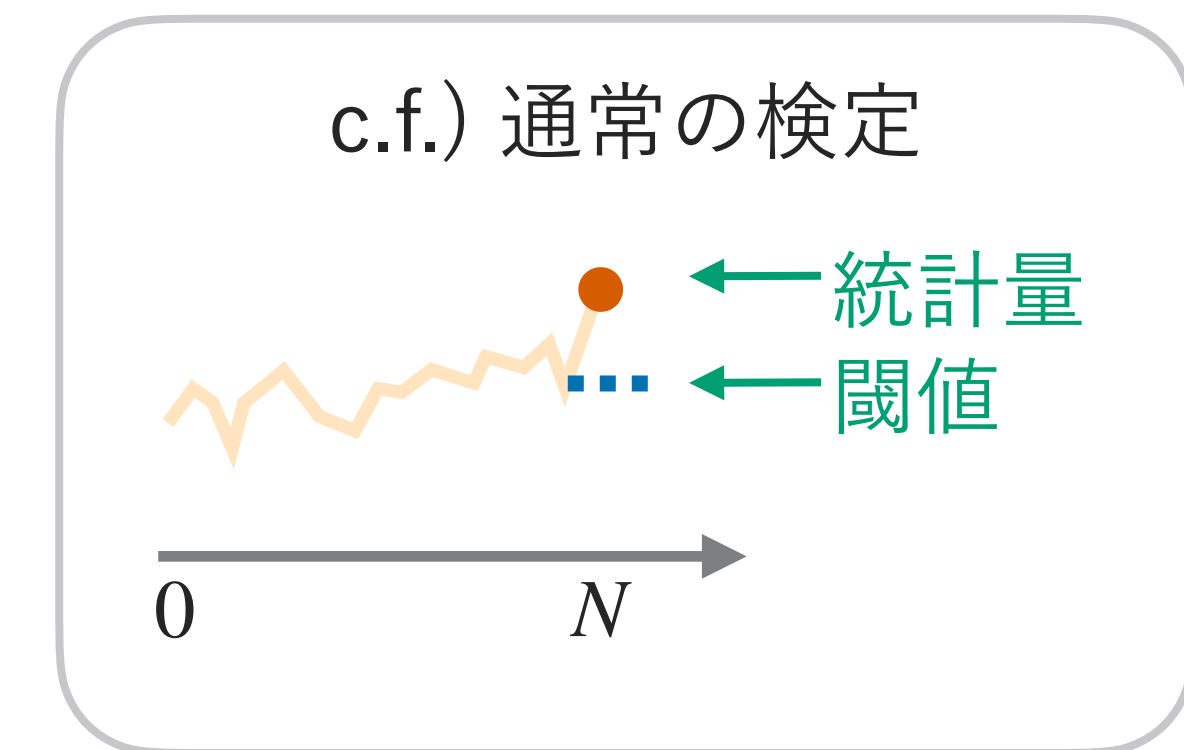
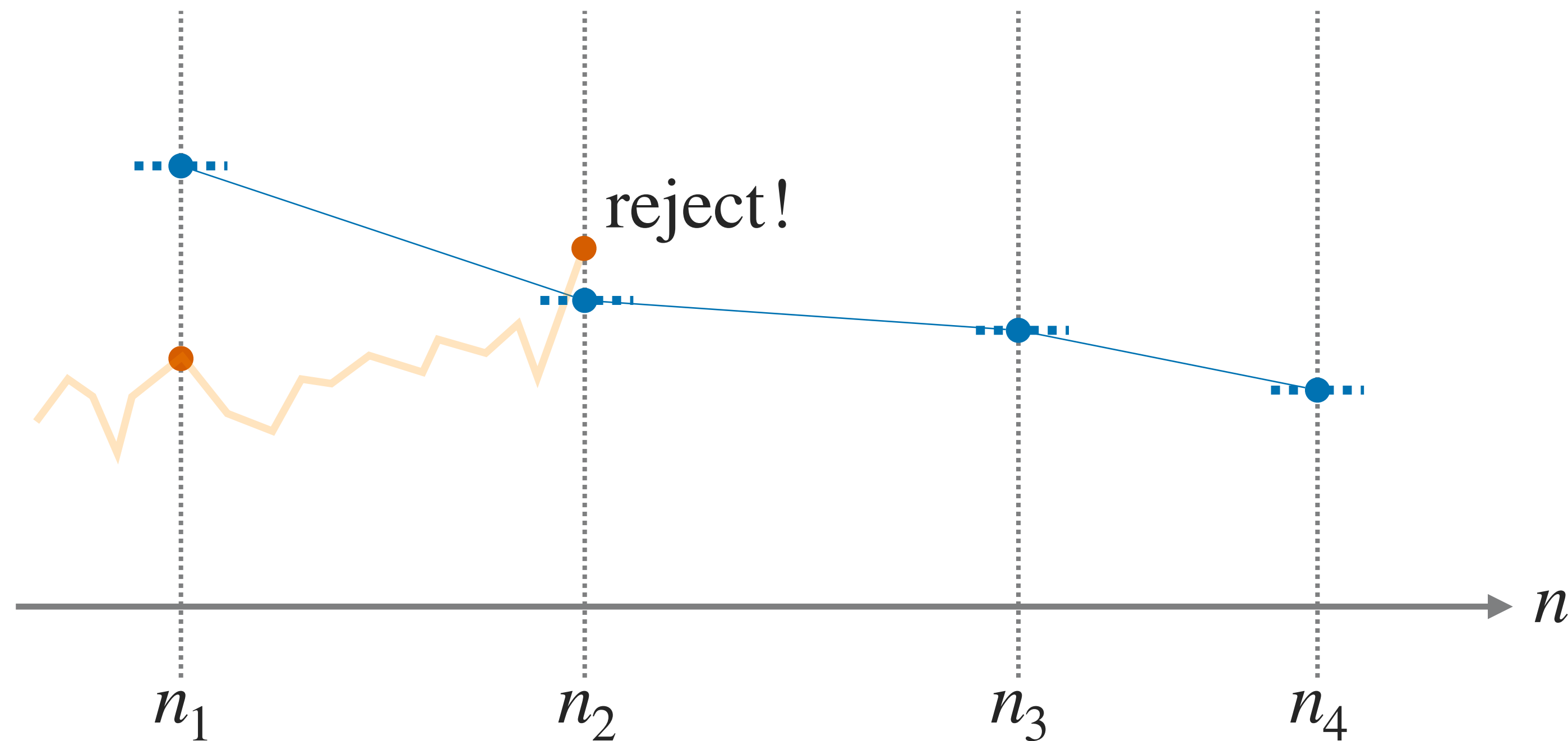
- この手続き全体として第一種の過誤率を α で抑えればよい

$$\mathbb{P}_{H_0}(\text{早めの検定で有意差 OR 最終検定で有意差}) \leq \alpha$$

- 基本の考え方はこれだけ！

群逐次検定による中間解析＞基本の考え方

具体的には，途中や最後に検定を行う際の，棄却の閾値を調整する



群逐次検定による中間解析＞基本の考え方

「閾値の調整」とはどういうことか、2ステップの場合で説明

- 記号を簡単にするために、検定統計量は Z で表し、片側検定の場合で説明する
 - つまり、 $Z > c \rightsquigarrow \text{reject}$ という形式の検定の場合で説明する
- 2ステップで行う検定（中間1回、最終1回）

$$\text{1st step} \begin{cases} Z_1 > c_1 \rightsquigarrow \text{reject} \\ Z_1 \leq c_1 \rightsquigarrow \text{continue} \end{cases} \xrightarrow[\text{(if continued)}]{\Rightarrow} \text{2nd step} \begin{cases} Z_2 > c_2 \rightsquigarrow \text{reject} \\ Z_2 \leq c_2 \rightsquigarrow \text{accept} \end{cases}$$

- このときの、第一種の過誤は、次のように分解できる

$$\mathbb{P}_0(\text{reject}) = \underbrace{\mathbb{P}_0(Z_1 > c_1)}_{\text{reject at 1st step}} + \underbrace{\mathbb{P}_0(Z_1 \leq c_1, Z_2 > c_2)}_{\text{continue at 1st, reject at 2nd}}$$

群逐次検定による中間解析＞基本の考え方

通常，次の手順で閾値を決めてゆく方法がとられる

- まずは，第一種の過誤率の各項それぞれに， $\alpha_1 + \alpha_2 = \alpha$ なる値を配分しておく

$$\underbrace{\mathbb{P}_0(Z_1 > c_1)}_{\leq \alpha_1} + \underbrace{\mathbb{P}_0(Z_1 \leq c_1, Z_2 > c_2)}_{\leq \alpha_2}$$

- まず第1項が不等式を満たすように c_1 を決め，続いて第2項が不等式を満たすように c_2 を決める

$$\mathbb{P}_0(Z_1 > c_1) \leq \alpha_1, \quad \mathbb{P}_0(Z_1 \leq c_1, Z_2 > c_2) \leq \alpha_2$$

- あとは，ここで決めた (c_1, c_2) を閾値として，2回それぞれの検定をすればよい

群逐次検定による中間解析＞基本の考え方

3回以上の中間解析の場合も，当然，同じ考え方が使える

- 一般には，次を満たすように閾値 $\{c_k\}_k$ を決めれば，第一種の過誤を抑えられる

$$\mathbb{P}_0(\exists k : Z_k > c_k) \leq \alpha$$

- 「帰無仮説が正しいときに，どこかで一度でも棄却が発生」を α で抑える

- 順番に検定を行なっていく，という構造を書き下して閾値を決めていく：

- $\mathbb{P}_0(Z_1 > c_1) \leq \alpha_1$

- $\mathbb{P}_0(Z_1 \leq c_1, Z_2 > c_2) \leq \alpha_2$

- $\mathbb{P}_0(Z_1 \leq c_1, Z_2 \leq c_2, Z_3 > c_3) \leq \alpha_3, \dots$

群逐次検定＞詳細



ここまでは概略. 実際に使う上では, 次の詳細を決める必要がある

- いつ中間解析をするか？
- どのように α の分配を決めるか？
- どのように先ほどの条件を満たす閾値を計算するか？

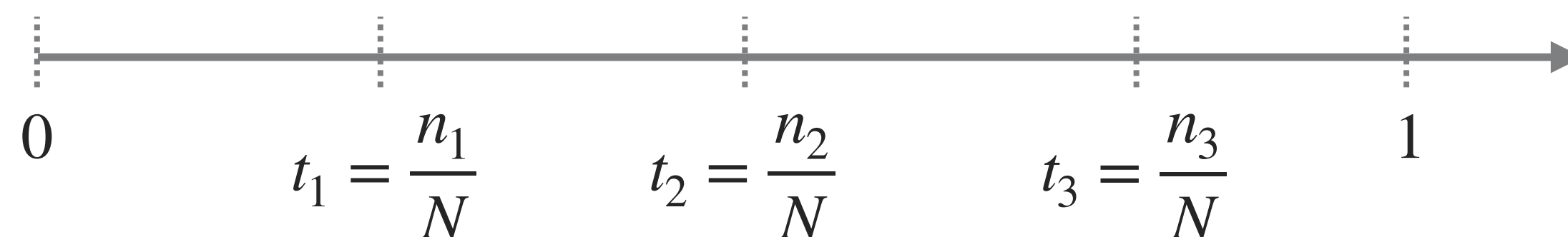
群逐次検定＞詳細＞STEP1 スケジュール

まずは、「いつ」中間解析をするのかを決める

- ここでいう「いつ」は、**情報時間 (information time)** で表す

最終時点のサンプルサイズ N に対して n データ集まっている時点なら、 $t := \frac{n}{N}$

- 「いつ時点で」検定をしようと考えているかを、最大サンプルサイズで割って $[0,1]$ の範囲の時間として表す



- 中間解析スケジュールの例：「 $t = (0.5, 0.75, 1)$ の3回にわたって検定」

- 記号：解析の回数は K で、解析実施タイミングは t で表す (t は必ず 1 を含む)

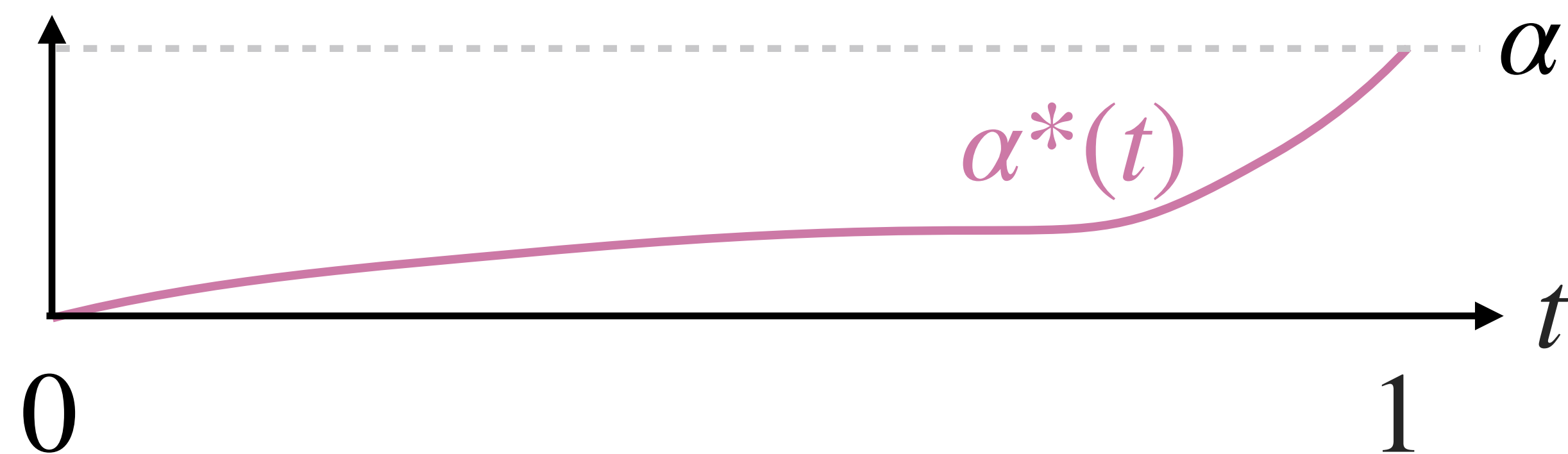
【用語】情報時間は、特に臨床試験での逐次検定での用語。感覚的に分かりやすい用語なので、本資料でも採用した。情報分数 (information fraction) ともいう。

群逐次検定 > 詳細 > STEP2 α の配分方法

続いて、スケジュールに対して α の配分を決める

■ Lan-DeMetsのアルファ消費関数法

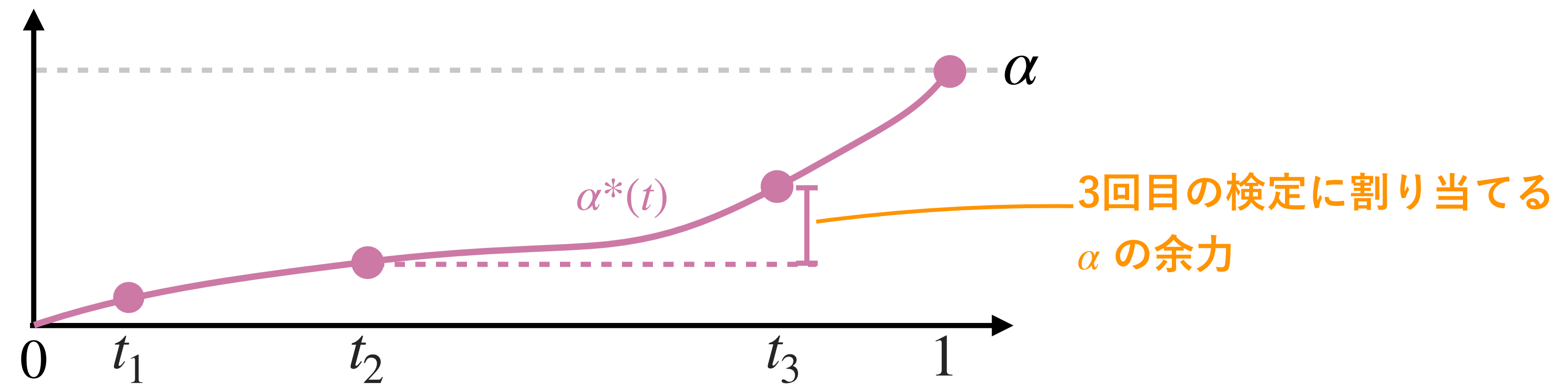
- 情報時間 t の関数である α -消費関数 (spending function) というものを用意
- $t \in [0,1]$ の関数 $\alpha^*(t)$ で、単調増加で、 $\alpha^*(0) = 0$, $\alpha^*(1) = \alpha$ を満たすもの



群逐次検定 > 詳細 > STEP2 α の配分方法

α の配分の仕方 (Lan-DeMetsのアルファ消費関数法)

- 検定を行いたい時点の $\{t_k\}_{k=1}^K$ に対し, それぞれに $\alpha_k := \alpha^*(t_k) - \alpha^*(t_{k-1})$ を配分

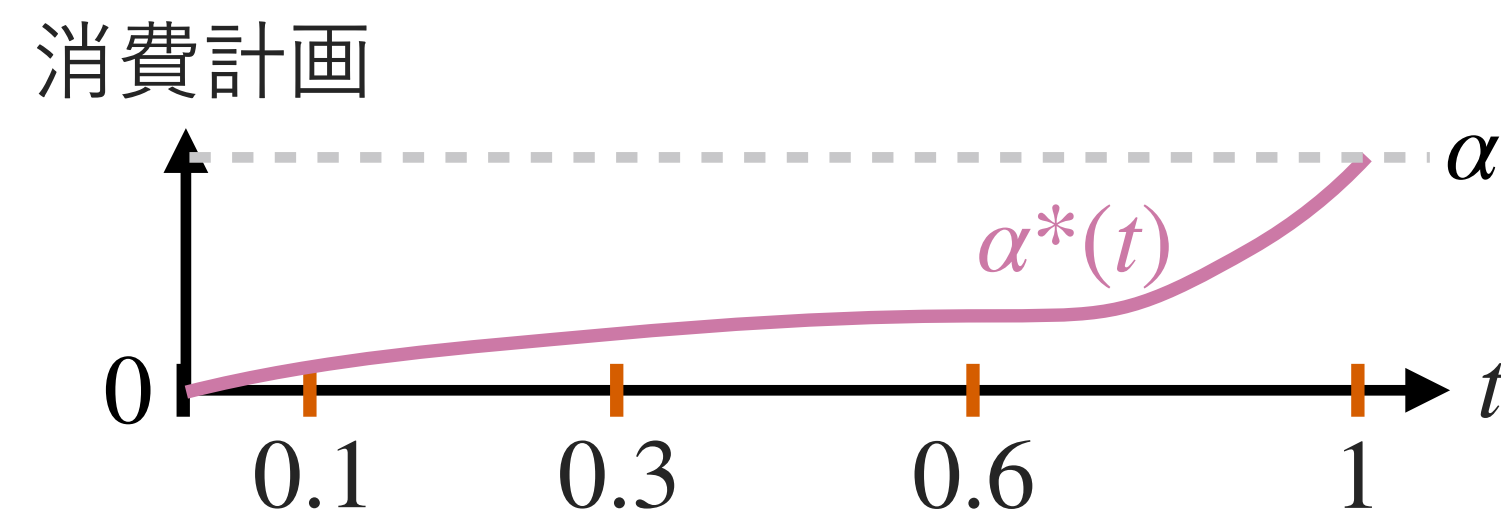


- これで計算した $\alpha_1, \dots, \alpha_K$ を割り当てとして, 閾値を1回目から順に決めていく

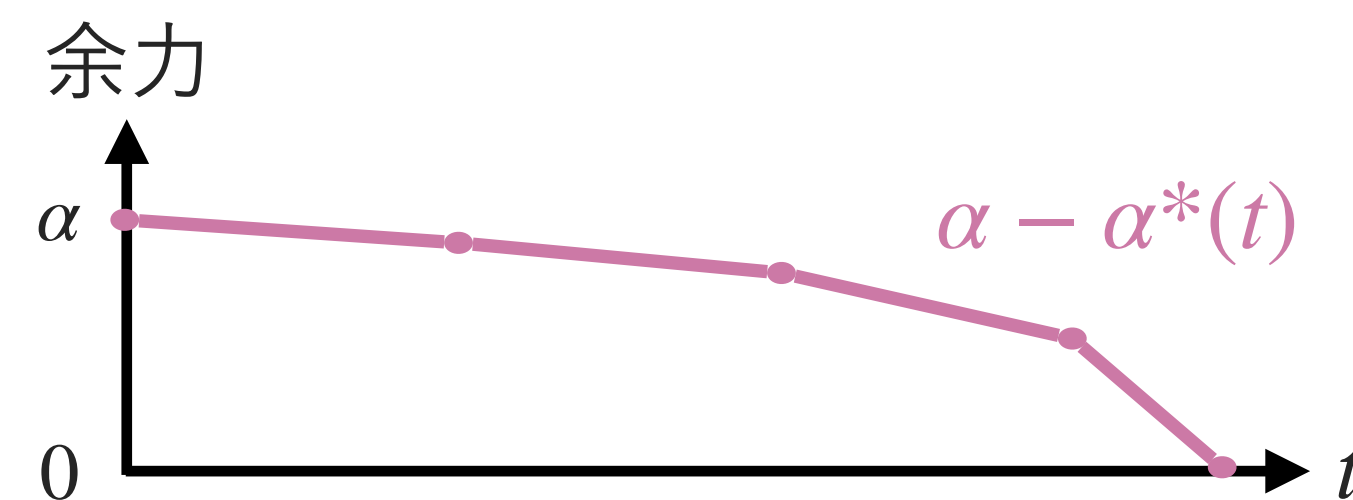
群逐次検定 > 詳細 > STEP2 α の配分方法

アルファ消費関数が持つ意味

- 各時点までに α をどれだけ消費済みにしようかという計画



- 裏を返せば $(\alpha - \alpha^*(t))$ 各時点でどれだけの α を残すかという温存計画でもある



群逐次検定 > 詳細 > STEP2 α の配分方法 > 具体例

よく知られた消費関数はいくつかある

- O'Brien–Fleming型（慎重，後半で消費）

$$\alpha^*(t) := 2 \left(1 - \Phi \left(z_{1-\alpha/2} \cdot t^{-1/2} \right) \right)$$

- Kim-DeMets型（冪乗型，パラメーター $\rho > 0$ で消費計画を調整可能）

$$\alpha^*(t) := \alpha \cdot t^\rho$$

Georgiev (2019) は、
← Kim-DeMets型の $\rho = 3$ をやや推奨.

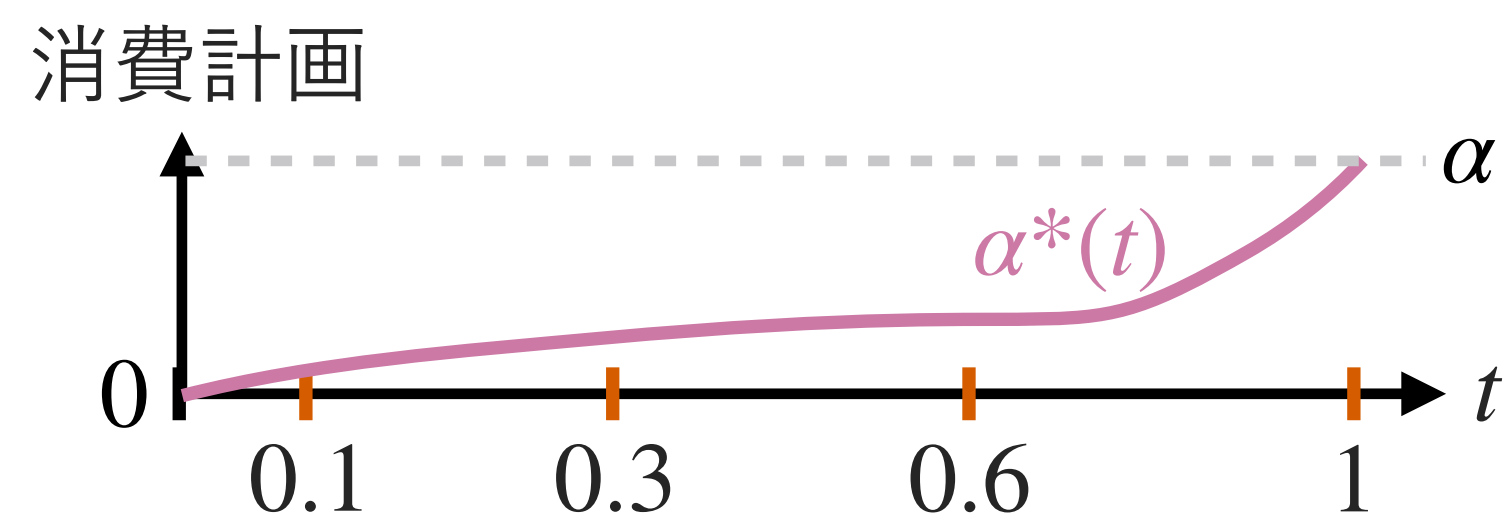
- Hwang–Shih–De Cani型（HSD型，パラメーター $\gamma \in \mathbb{R}$ が小さいほど余力温存）

$$\alpha^*(t) := \alpha \cdot \frac{1 - e^{-\gamma t}}{1 - e^{-\gamma}}$$

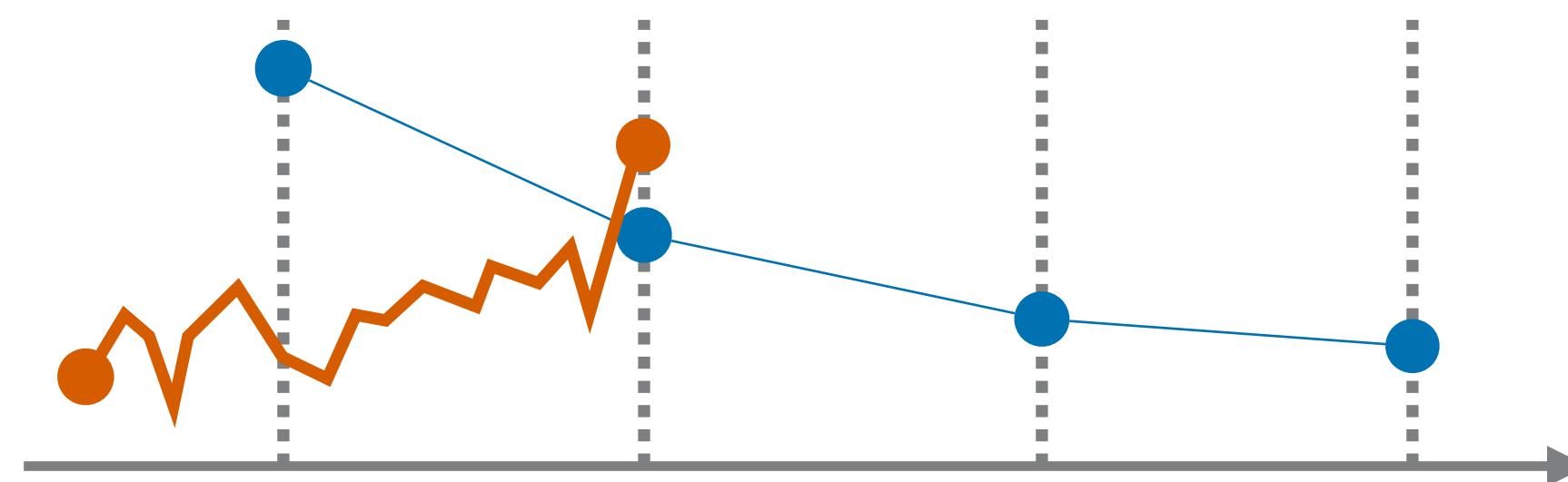
群逐次検定 > 詳細 > STEP2 α の配分方法 > 具体例

α -消費関数の選択は，手法の動作特性を決める重要な要素

- 最初のうちは α を温存し，後半で大きく消費するような形状にしておくとい



- そうすることで，「初速が良い場合に限って早めに検出」（早い閾値は厳しめ）という直感に合った設計になる

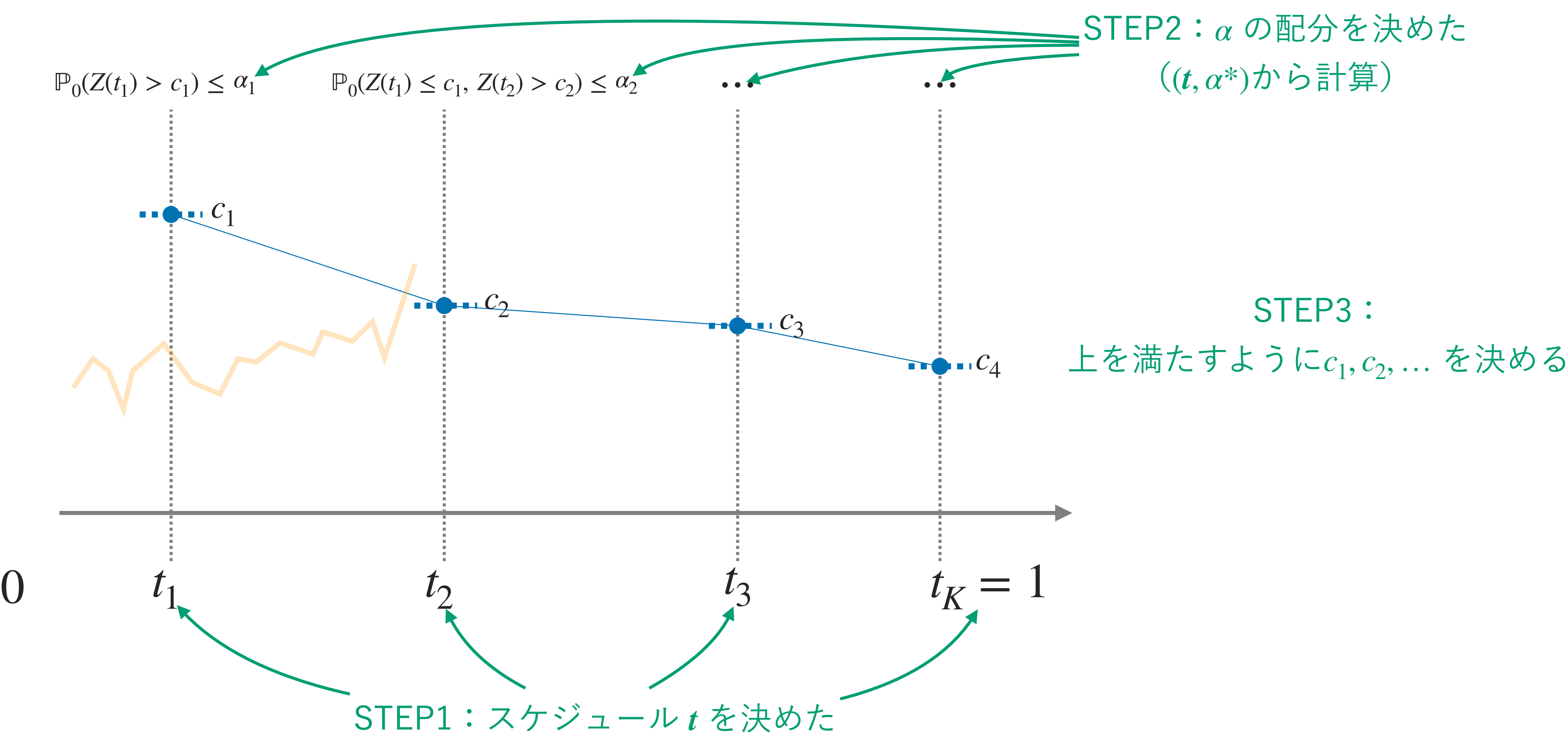


- それと同時に，最終回の閾値が，中間解析無しの場合に近い値になる（＝従来の設計と比べて，さほど失うものなく中間解析を追加できる）

【補足】 さらに，結果として，必要な最大サンプルサイズも，さほど増えずに済むことになる。

群逐次検定 > 詳細 > STEP3 検定統計量と閾値

スケジュール t , アルファ消費関数 $\alpha^*(t)$ が決まったら, 最後に**閾値**を決める

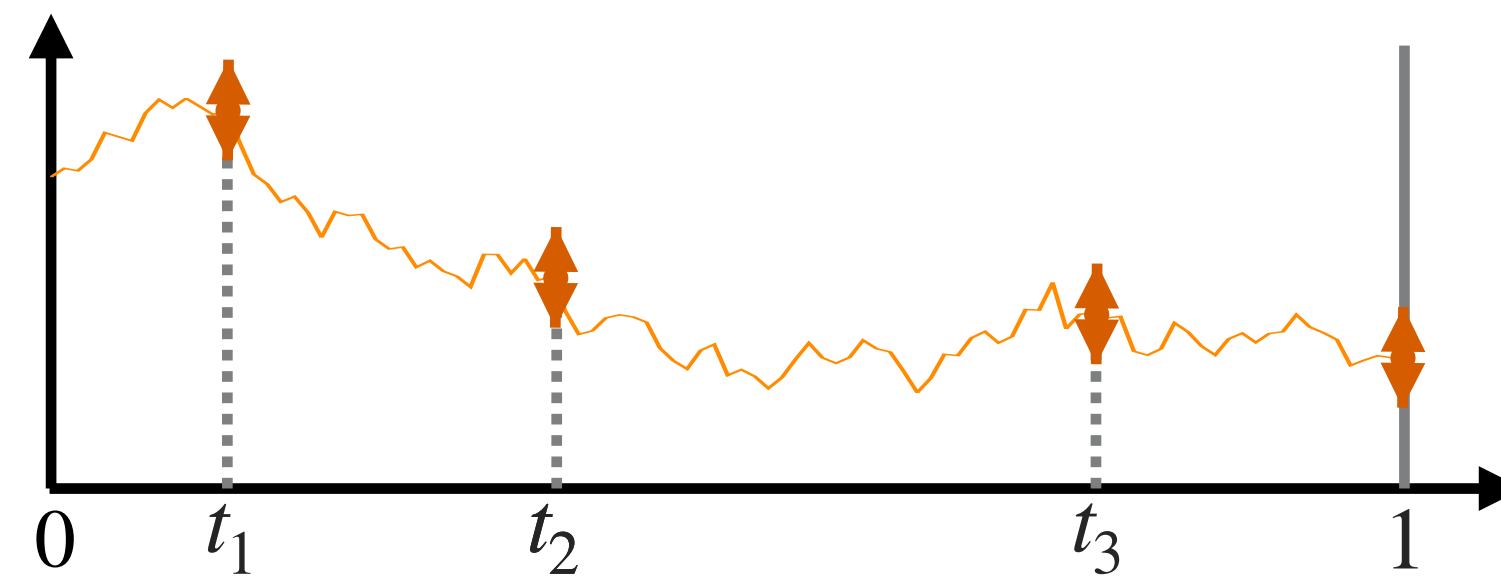


群逐次検定 > 詳細 > STEP3 検定統計量と閾値

閾値を決めるには，統計量の同時分布が必要

- 帰無仮説における，統計量の「時点間の」同時分布が分かっている必要がある

$$\mathbb{P}_0(Z(t_1) \leq a, Z(t_2) \leq b, Z(t_3) \leq c)$$



- 具体的には，例えば，同時分布に基づいて次のような問題を解くことになる

次を満たす最小の c_3 を求めよ： $\mathbb{P}_0(\underbrace{Z(0.5) \leq 3.6}_{Z(t_1) \leq c_1}, \underbrace{Z(0.75) \leq 2.1}_{Z(t_2) \leq c_3}, \underbrace{Z(0.9) > c_3}_{Z(t_3)} \leq \underbrace{0.5}_{\alpha_3})$

【補足】 同時分布が必要と言ったが，あらゆる同時分布が必要というわけではなく，途中までを条件付けた1つ先の条件付き分布などが分かれば十分。

群逐次検定＞詳細＞STEP3 検定統計量と閾値

どういう検定統計量なら，同時分布が分かるか？

- データ量がそれなりにある場合，関数版中心極限定理での近似がよく使われる

$$\text{(帰無仮説において)} \quad \begin{pmatrix} Z(t_1) \\ Z(t_2) \\ \vdots \\ Z(t_K) \end{pmatrix} \sim \mathcal{N}(\mathbf{0}, \Sigma) \quad \text{ここで, } \Sigma_{i,j} = \sqrt{\frac{t_i}{t_j}} \quad (i \leq j)$$

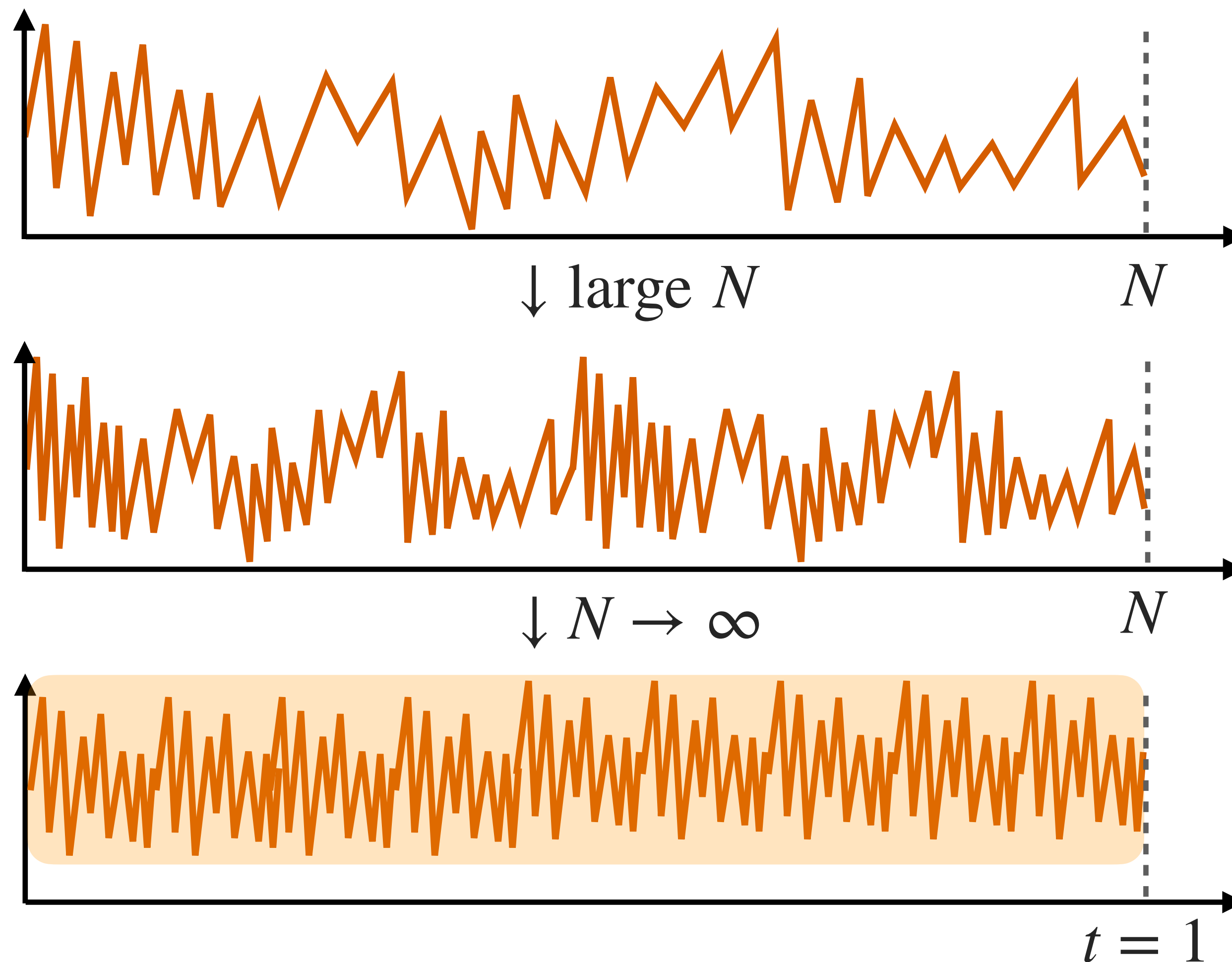
- 関数版中心極限定理：A/Bテストでよく使われる Z 統計量などの場合に成立
- 対立仮説 δ においては

$$\begin{pmatrix} Z(t_1) \\ Z(t_2) \\ \vdots \\ Z(t_K) \end{pmatrix} \sim \mathcal{N}(\boldsymbol{\mu}_\delta, \Sigma) \quad \text{ここで, } \boldsymbol{\mu}_\delta = \delta \cdot \begin{pmatrix} \sqrt{I(t_1)} \\ \vdots \\ \sqrt{I(t_K)} \end{pmatrix}, \quad \Sigma_{i,j} = \sqrt{\frac{t_i}{t_j}} \quad (i \leq j)$$

【補足】 ここで $I(t)$ は，統計量 Z に対応する情報量。言い換えると，漸近分布が上記になるような I が存在するような Z が，この方法で扱える対象となる。

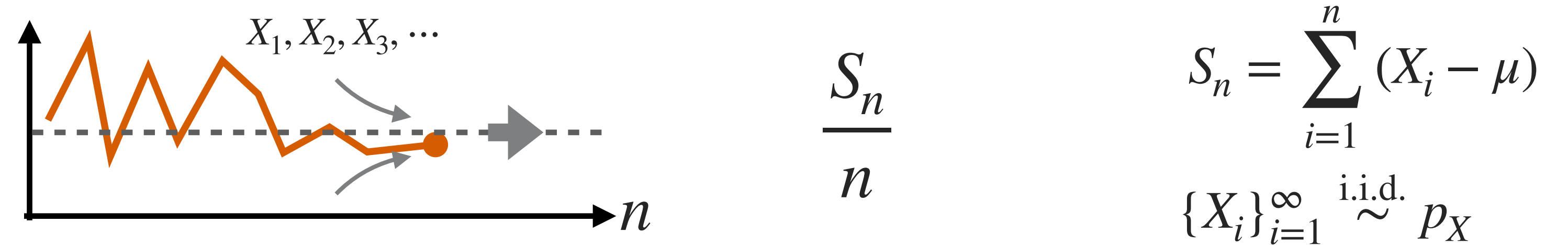
群逐次検定 > 詳細 > STEP3 検定統計量と閾値 > 近似

この場合の中心極限定理：横幅固定でデータ増大→とあるガウス過程に漸近

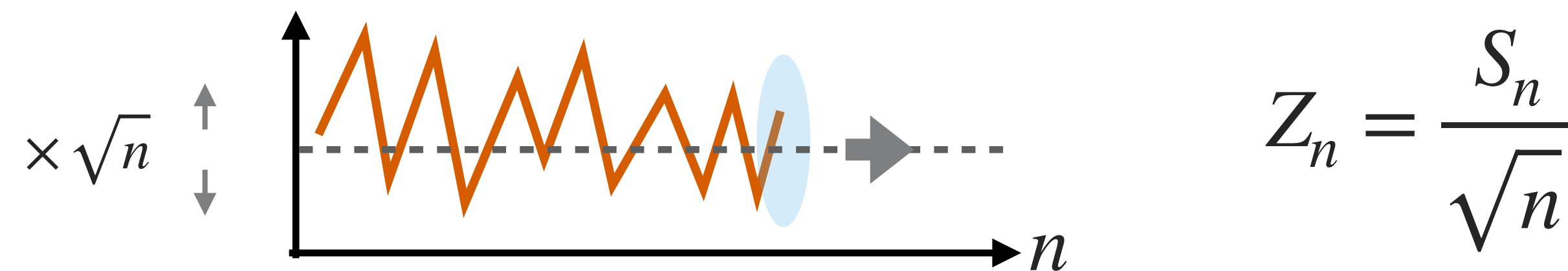


群逐次検定 > 詳細 > STEP3 検定統計量と閾値 > 近似

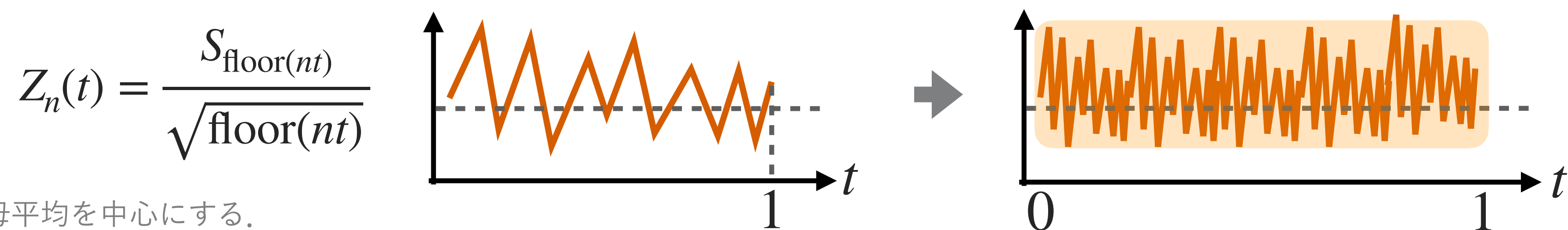
- データをだんだん増やしていくと、平均値は一定値（母平均）に収束していく



- これを縦に $\sqrt{n}$ 倍に拡大すると、最後のあたりは中心極限定理でガウス分布に収束



- さらに、横幅を1に規格化して、そのまま $n \rightarrow \infty$ すると、あるガウス過程に収束



【補足】 μ は母平均。拡大は母平均を中心にする。

これまでのあらすじ

- 後続の検定は，途中のステージで停止しなかった場合にのみ実施される！
- 途中で棄却＝停止する意図を持って覗き見るということは……
 - 1回目の検定に引っ掛からなかった場合のみ2回目の検定がある
 - 1・2回目の検定に引っ掛からなかった場合のみ3回目の検定がある
 - 1・2・3回目の検定に引っ掛からなかった場合のみ4回目の検定がある
- 検定タイミングは「情報時間」で指定する（サンプル収集進捗率）
- アルファ消費関数を選んでおくことで，第一種の過誤の配分計画を決める
- 閾値は中心極限定理でガウス過程を近似モデルとして計算するのが主流

群逐次検定＞詳細＞使用上の注意

手続き上，解析スケジュールは事前に決めていなくてもよい

- 回数 K も，タイミング t も，必要に応じて途中で変えてもOK
 - 最終サンプルサイズは，今回紹介した手法の範囲では，途中変更不可（そこまで織り込んだ検定としてデザインする必要があるため）
 - 統計量の値（結果指標）を見ながら解析タイミングを決めるのは不可
- 実際に検定する情報時刻は，計画上のタイミング t と少しだけズレる
 - 「実際の情報時刻を計算」→「アルファ消費関数で今回の α の配分を決定」→「これまでの閾値の履歴とあわせて，今回の閾値を決定」とする必要がある

群逐次検定＞詳細＞使用上の注意

次のように検定タイミングは計画から多少はズレても問題ない

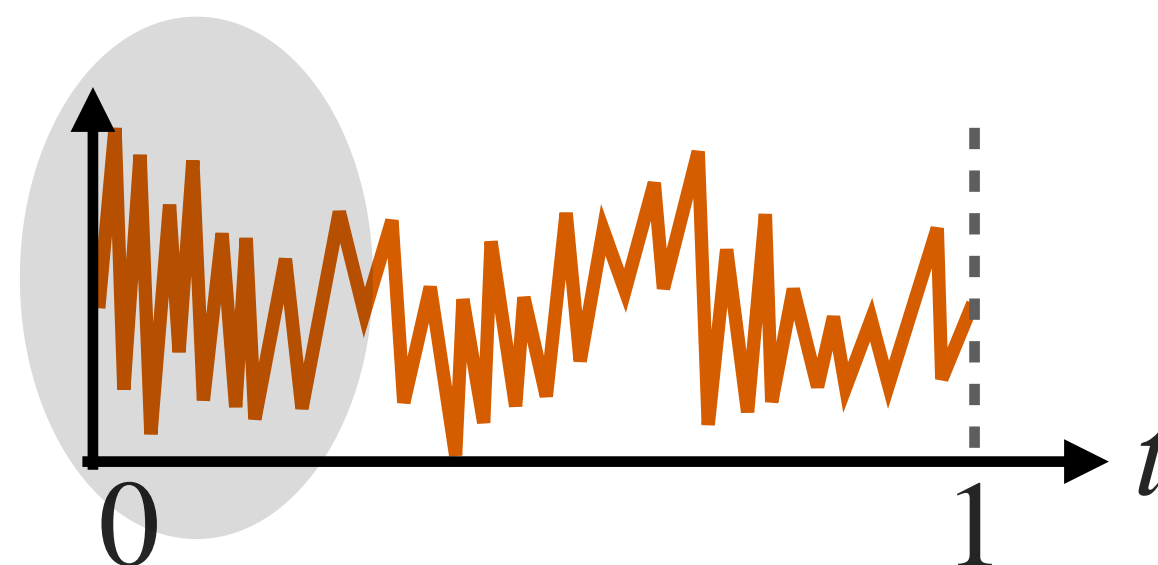
- 🙋 「予定タイミングを少し過ぎてしまったが、今日解析しよう」はできる
- 🙋 「思ったほどデータは集まらなかったが、今日解析してみよう」もできる
- 🙅 「初速がいいから、近いうちに何回か中間解析しよう」は不可
 - 検定結果や指標の値に基づいてスケジュールを変えると、依存関係が生まれてしまい統計的保証が破綻するため
- 🙅 計画時のスケジュールから大きくズレた中間解析をしてしまうのは不可
 - 最大サンプルサイズ設計はタイミング t も考慮して行われるため（結果的に集めすぎ・集めなさすぎになりうる）

【補足】途中で結果を見てスケジュール変更する、という手続きを丸ごと考慮に入れて計画できるなら、そのような適応的変更も可となる。これは適応的デザインの領域。

群逐次検定＞詳細＞使用上の注意

早すぎる検定タイミングは、近似が効かないのでNG

- 左端の $t = 0$ に近すぎる、早すぎるタイミングには中間解析を入れないほうがよい



- 例) 仮に $N = 10000$ ほど集めても、 $t = 0.001$ で検定しようとするすると、それはまだ10件しか集まっていないタイミングになる
- $N \rightarrow \infty$ とする中で、だんだんと中身が詰まっていくので、（左のほうも含めて）近似が効いていく
- どうせ検定しても棄却できないだろうから無駄になる、という意味でも避けたい

群逐次検定＞詳細＞使用上の注意

情報時間の計り方は，サンプルサイズの進捗率ではない場合もある

- イベントデータ（生存解析）の場合には，全体の観測数ではなくイベント数
 - 一般には「統計量の根拠にあるマルチンゲールの予測可能二次変分」に由来
 - ▶ スコア関数に基づく統計量の場合は，パラメーターのFisher情報量に由来

群逐次検定 > 具体例 > 2値変数の場合

2標本・2値変数での、成功確率の差異の検定（よくあるA/Bテスト）

- $\{X_i\}_{i=1}^N \stackrel{\text{i.i.d.}}{\sim} p_A, \{Y_i\}_{i=1}^N \stackrel{\text{i.i.d.}}{\sim} p_B$: それぞれ A/B の逐次的標本 (0/1-値)
- 標準化検定統計量

$$Z_k := \frac{\sum_{i=1}^{n_k} (Y_i - X_i)}{\sqrt{2n_k \cdot \hat{\sigma}_k^2}}$$

(ここで $\hat{\sigma}_k^2$ は、第 k 解析までのサンプルによるプール分散推定値 $\hat{p}_k(1 - \hat{p}_k)$)

- 情報時間はもちろん, $t_k := \frac{n_k}{N}$

【参考】 Chapter 12 in Jennison, C., & Turnbull, B. W. (2000). Group Sequential Methods with Applications to Clinical Trials. Chapman and Hall/CRC.

【補足】 $\hat{p}_k := \frac{1}{2n_k} \sum_{i=1}^{n_k} (X_i + Y_i)$.

群逐次検定＞具体例＞2値変数の場合

2標本・2値変数での、成功確率の差異の検定（よくあるA/Bテスト）

- $\{X_i\}_{i=1}^N \stackrel{\text{i.i.d.}}{\sim} p_A, \{Y_j\}_{j=1}^M \stackrel{\text{i.i.d.}}{\sim} p_B$ ：それぞれ A/B の逐次的標本（0/1-値）
 - 不均一な場合（最終サンプルサイズは N, M ，第 k 解析時点では n_k, m_k とする）
- 標準化検定統計量

$$Z_k := \left(\frac{1}{m_k} \sum_{j=1}^{m_k} Y_j - \frac{1}{n_k} \sum_{i=1}^{n_k} X_i \right) \cdot \sqrt{\hat{I}_k} \quad \left(\text{ここで } \hat{I}_k := \frac{n_k m_k}{n_k + m_k} \cdot \frac{1}{\hat{p}_k(1 - \hat{p}_k)} \right)$$

- $\theta = p_B - p_A$ の情報量が $I_k := \frac{n_k m_k}{n_k + m_k} \cdot \frac{1}{p(1 - p)}$ になり，情報時間は $t_k := \frac{I_k}{I_K}$ となる．

【参考】 Section 3.6.2 in Jennison, C., & Turnbull, B. W. (2000). Group Sequential Methods with Applications to Clinical Trials. Chapman and Hall/CRC.

【補足】 ここで p は，帰無仮説において $p_A = p_B =: p$ である局外パラメーターであり， $\hat{p}_k := \frac{\sum_{i=1}^{n_k} X_i + \sum_{j=1}^{m_k} Y_j}{n_k + m_k}$ はそのプール推定量．

群逐次検定＞無益な場合の中止



「見込みなし」の場合の早期停止は？

■ Efficacy による停止（無益中止）

- 明らかに結果が良いならば早めに止めて全展開したい

■ Futility による停止（無益中止）

- 見込みがないならば早めに止めて、他のことにリソースを使いたい
- 逆に劣化しているなど、有害な場合もこちらに含まれる

群逐次検定＞無益な場合の中止

無益による中止の方式は，次の2種類から選べる：中止にコミットするか否か

- 拘束的（binding）な無益判定

- 中止することにコミットする
- それにより，有益性のための閾値を少し緩和できる！（途中で中止する理由が増えれば，第一種の過誤は減るため）

- 非拘束的（non-binding）な無益判定

- 中止にコミットしない（＝止めても良い，止めなくても良い．柔軟な判断で）
- その分，検出力を保つためのサンプルサイズが少し嵩む傾向

群逐次検定＞設計

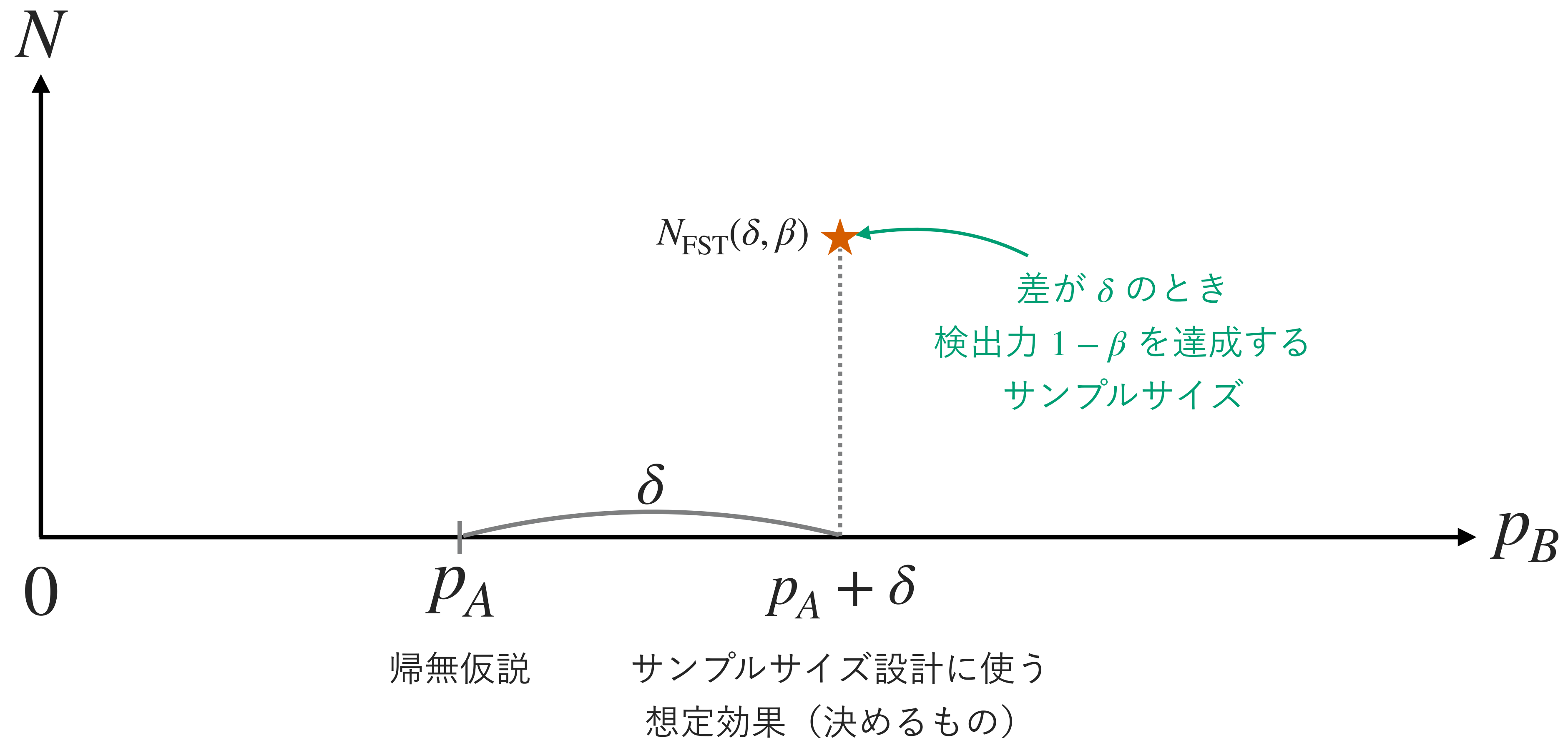


ここまでが検定の手続きの話

- ここからは、サンプルサイズ設計など、この検定の設計に関する話
 - スケジュール t の選び方（←ここでは扱わない）
 - アルファ消費関数 $\alpha^*(t)$ の選び方（←ここでは扱わない）
 - 最大サンプルサイズ N の設計（←ここで扱う）
 - ▶ 平均サンプルサイズという考え方
 - ▶ 検出可能な効果量についての考え方も、固定サンプル検定の場合と少し変わる

群逐次検定 > 設計 > サンプルサイズ設計

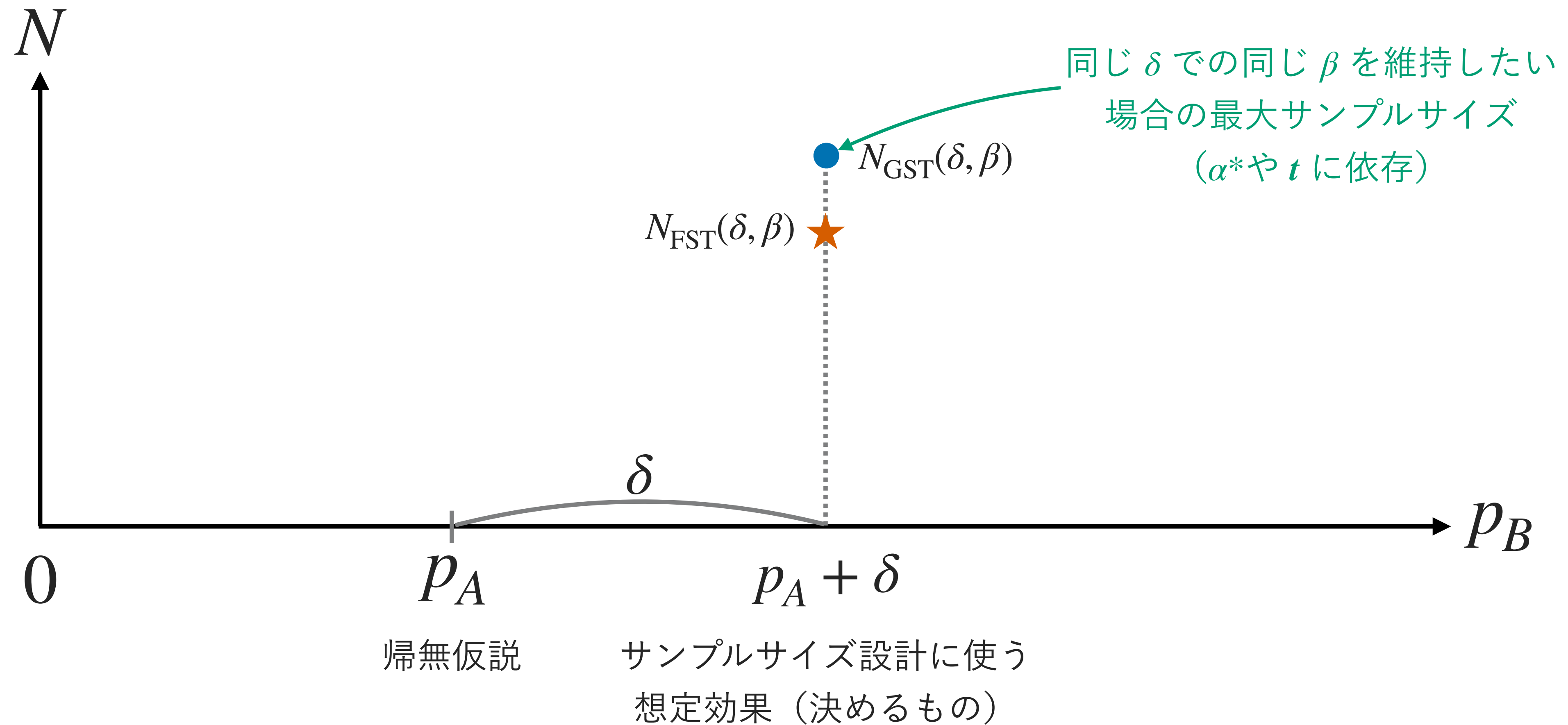
通常の固定サンプル検定（fixed sample test, FST）の場合



【補足】 サンプルサイズ設計に使う想定効果は、通常、検出したい最小の効果に設定する。真の効果が大きいは集め過ぎかもしれないが、大は小を兼ねるため。

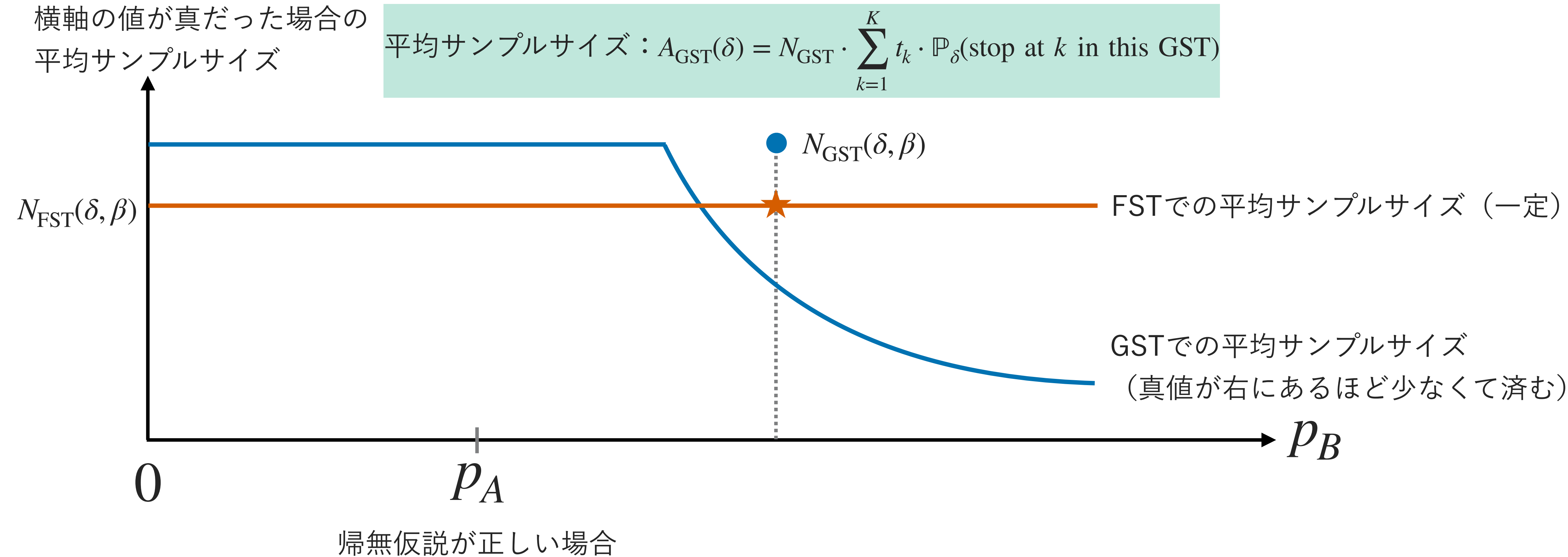
群逐次検定 > 設計 > サンプルサイズ設計

群逐次検定で、同じ δ で同じ β を維持するには最大サンプルサイズが微増



群逐次検定 > 設計 > サンプルサイズ設計

その代わり，うまくいけば平均でのサンプルサイズは減少



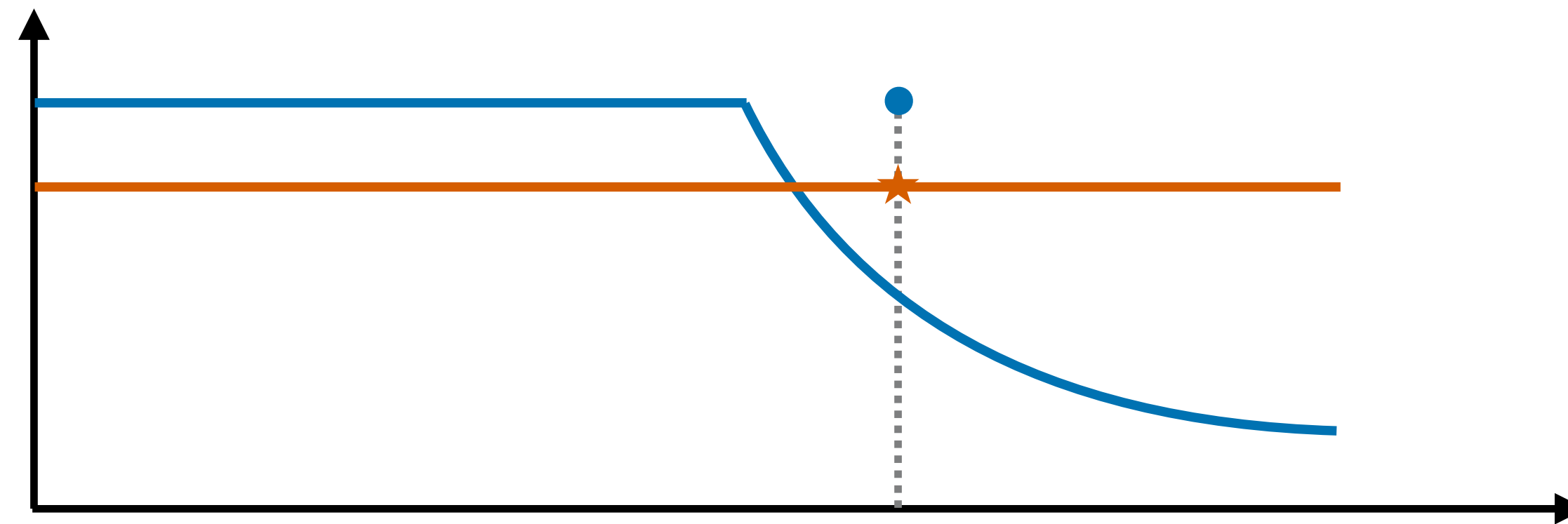
【補足】 ここでは片側検定だけ説明しているので，左側ではサンプルサイズが減らないが，無駄削減の早期停止も追加すると左側もサンプルサイズが減る。
【補足】 先ほどの図は設計のため「想定値1点とその場合の計画」を描いていたが，この図では「横軸の値が実際のパラメーターだった場合の理論値」を描いている。

群逐次検定＞設計＞サンプルサイズ設計

FSTとGSTでの動作特性の違い：効果が大きい場合に，ちゃんと得をできる

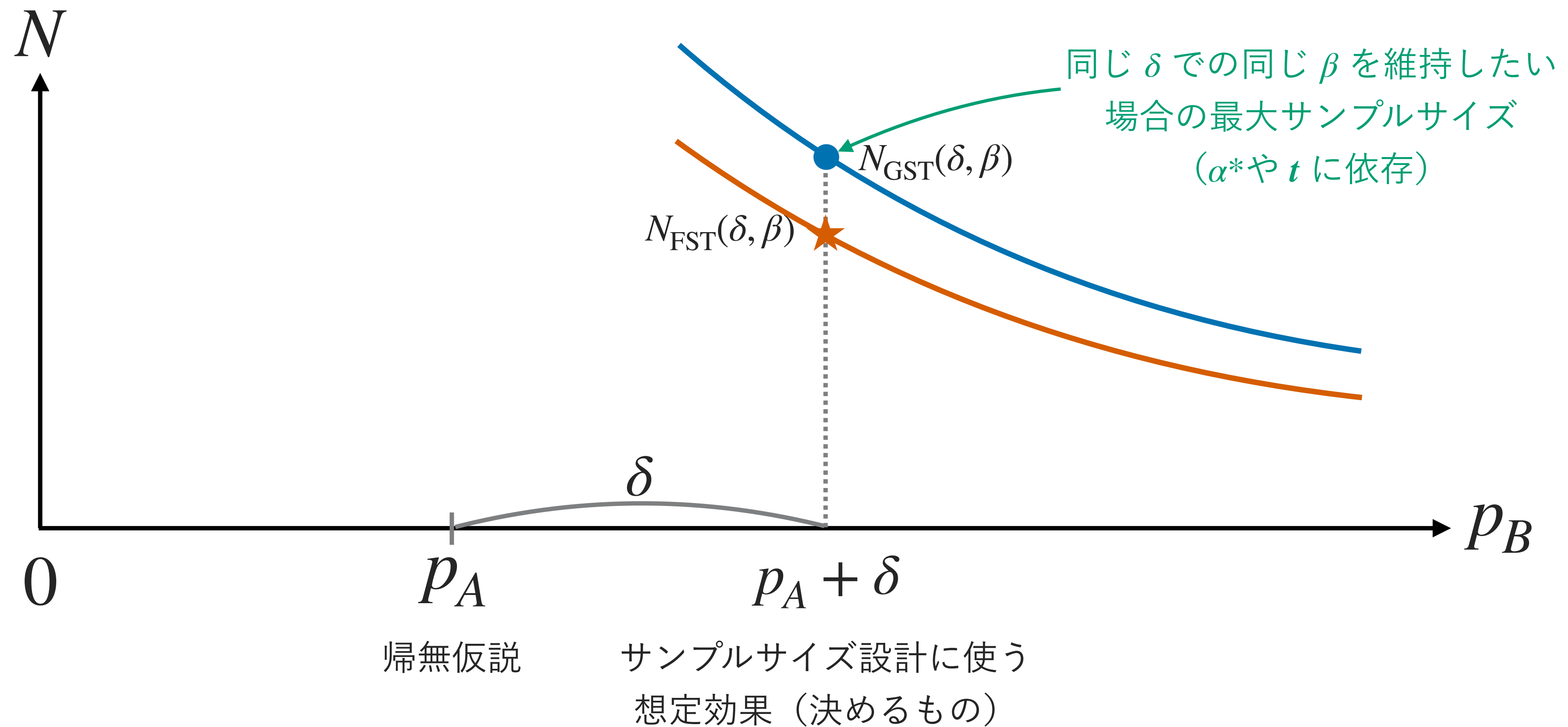
■ 想定よりも効果が大きかった場合

- FSTでは，サンプルサイズは固定値なので，ひたすら必要数を集めるだけ
- GSTでは，早めの解析で止まる確率が高くなり，平均サンプルサイズが減る
- 「思っていたより効果があった」場合を活かせる（早々に結果が出るので）



群逐次検定 > 設計 > サンプルサイズ設計

群逐次検定で、同じ δ で同じ β を維持するには最大サンプルサイズが微増



群逐次検定 > 設計 > サンプルサイズ設計 > 検出力への影響

微増で済む理由： $\alpha^*(t)$ の設計によっては、検出力はあまり落ちない

- 検出力は、最後の1回の検出力以上になる
- つまり、 $\mathbb{P}_1(Z(1) \geq c_K)$ と $\mathbb{P}_1(Z(1) \geq c_{\text{FST}})$ の比較だけすればよい。
 - 最後の1回の閾値 c_K が c_{FST} に近ければ、あまり検出力は落ちない
 - 「後のほうに余力を残す」形状の $\alpha^*(t)$ を使えば、あまり検出力が落ちない
 - $\alpha^*(t)$ の設計に妙がある

【補足】 ここで、 c_{FST} は固定サンプル検定の場合の、 α などから決められる閾値。

群逐次検定＞設計＞サンプルサイズ設計



これを踏まえて、サンプルサイズの設計はどうか？

- 検出力が若干落ちることを気にしないならば、サンプル数の設計は通常通り、固定サンプル検定の設計と同じにしてもよい
- 気にするならば,
 - 「想定している効果量において（＝対立仮説の中から1つ選んで）」 「最大サンプルサイズにおける検出力が、設定値より大きくなるように」 サンプルを設計する

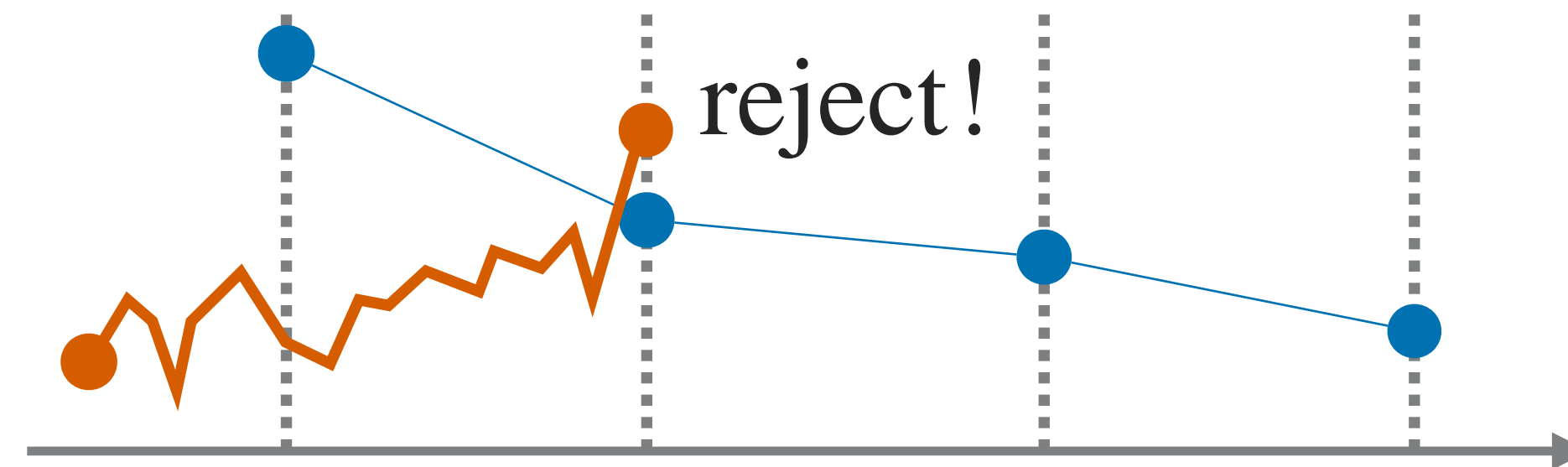
今回のまとめ

群逐次検定による中間解析＞まとめ＞使い方



試してみませんか？

- 今、A/Bテスト基盤があるなら、そのまま使える
 - 今と同じ統計量を中間でも計算するだけ。ただし、判定の閾値を変える



- これからA/Bテストを計画する（まだソースコードはない）ならば……

群逐次検定による中間解析＞まとめ＞使い方

ライブラリを作り始めています



Early signs, faster decisions.

```
> pip install earllysign
```

※まだ建設中です

群逐次検定による中間解析＞まとめ



気になったという方，ご相談ください

- まずは，過去のA/Bテストでのデータを使って，リプレイで検証してみましょう
- 今回紹介していない発展形や，別の特性の手法も色々あるので機会があれば紹介
 - Group-sequential系で，目標とするKPIの早期停止を仕込んでおく
 - ▶ 上振れを考慮して早めに1回入れておくなど
 - Anytime-valid系で，ガードレールは常時監視
 - ▶ 検定力は下がるが，劇的な悪化をタイムリーに検知
 - 途中で計画変更する手法（特にサンプルサイズ変更や，段階的候補脱落など）

もっと学ぶための資料案内



参考資料はこちら

- Spotify R&D Blog <https://engineering.atspotify.com/2023/03/choosing-sequential-testing-framework-comparisons-and-discussions>
 - ここで説明した手法は， Spotifyで実践されている
- gsDesign Technical Manual
 - <https://keaven.github.io/gsd-tech-manual/>
 - 解説が非常に豊富

もっと学ぶための資料案内



代表的な教科書の系譜

- Jennison, C., & Turnbull, B. W. (2000). Group Sequential Methods with Applications to Clinical Trials. Chapman and Hall/CRC.
 - 「古典的な」群逐次デザインを体系化した教科書
 - 消費関数法など、中間解析を可能にするための統計的理論基盤
 - 群逐次デザインでは、試験計画は原則として固定
- Wassmer, G., & Brannath, W. (2016). Group Sequential and Confirmatory Adaptive Designs in Clinical Trials. Springer International Publishing. <https://doi.org/10.1007/978-3-319-32562-0>
 - さらに進んで検証的適応デザインを体系化した教科書（群逐次デザインも包括）
 - 統合検定原理や条件付き過誤関数など、適応的計画を可能にするための統計的理論基盤
 - 途中で計画変更（特にサンプルサイズ変更や、段階的候補脱落など）を可能にする理論を扱っている

中間解析＞補足



詳しい人は、ボンフェローニ法やシダック法で補正すれば？と思うかも

- 使うこと自体はできるが、それらは過度に保守的
 - ボンフェローニ法は、全く同じデータを、何回でも、どのように使い回しても、正当化が崩れない（つまり、それほど悲観的な前提で作られており、保守的）
 - 保守的というのは、なかなか有意判定を出さないということ（検出力が低い）
- 今回はBonferroni法よりもずっと楽観的な手法の余地がある
 - 「だんだんデータが蓄積されていく」という状況なので、その構造を使える
 - 言い換えれば、Bonferroni法の想定よりは「使い回しの度合いが低い」状況
 - それが今回のトピック